UNIVERSIDAD CATÓLICA DE EL SALVADOR

FACULTAD DE INGENIERÍA Y ARQUITECTURA

INGENIERÍA EN SISTEMAS INFORMÁTICOS

"La Ciencia sin Moral es Vana"

TEMA:

AVANCE DE PROPUESTA DE DESARROLLO DE UNA APP

ASIGNATURA:

DESARROLLO DE APLICACIONES MÓVILES II

SECCIÓN:

A

PRESENTADO POR:

JOSÉ EDUARDO RAMOS LÓPEZ

DOCENTE:

ING. JOSÉ RAÚL RODRÍGUEZ FIGUEROA

SANTA ANA, SANTA ANA, FECHA DE ENTREGA (18/02/2026)

Contenido

1. OBJETIVOS DE LA APP 1
2. VALOR DIFERENCIAL 1
3. ESTADO DEL ARTE 1
4. REQUISITOS FUNCIONALES 2
5. CASOS DE USO DE LA APP 2
6. DEFINICIÓN DEL STACK TECNOLÓGICO 3
6.1 Stack Tecnológico 3
6.2 Arquitectura Propuesta (MVC) 3
6.3 Arquitectura General del Sistema 4
6.4 Persistencia de Datos 5
7. DISEÑO DE INTERFAZ (UI) Y MAPA DE FUNCIONAMIENTO 5
DISEÑO DE INTERFAZ (UI) 5
MAPA DE FUNCIONAMIENTO 6
BASE DE DATOS 7

1. OBJETIVOS DE LA APP

Objetivo general:

Desarrollar una aplicación móvil que permita a los usuarios explorar, buscar y visualizar información de películas de manera sencilla e interactiva.

Objetivos específicos:

- Mostrar películas populares
- Permitir búsqueda de películas
- Ver detalles de cada película
- Mostrar calificaciones y descripciones
- Guardar películas favoritas

2. VALOR DIFERENCIAL

La aplicación se diferencia porque:

- Interfaz simple y moderna
- Rápida navegación
- Permite guardar favoritos
- Muestra información clara de películas

A diferencia de otras aplicaciones, será más ligera y fácil de usar.

3. ESTADO DEL ARTE

Se analizó la aplicación:

- Netflix
- Disney

Fortalezas:

- Buen diseño
- Consumo de API
- Interfaz moderna

Debilidades:

- Requiere internet
- Puede ser compleja

Justificación:
Nuestra APP busca ser más sencilla, clara y accesible para el usuario.

4. REQUISITOS FUNCIONALES

La aplicación permitirá:

- Ver lista de películas
- Buscar películas
- Ver detalles
- Guardar favoritos
- Eliminar favoritos

5. CASOS DE USO DE LA APP

A continuación, se describen las interacciones principales que el usuario tendrá con el sistema para cumplir con los objetivos planteados:

• Caso de Uso 1: Explorar Catálogo de Películas

Actor: Usuario.

Descripción: Al iniciar la aplicación, el usuario visualiza automáticamente una lista actualizada con las películas más populares y tendencias del momento, permitiendo una navegación rápida por el contenido principal.

• Caso de Uso 2: Buscar Película

Actor: Usuario.

Descripción: El usuario ingresa el título o palabras clave en la barra de búsqueda. La aplicación procesa la solicitud y devuelve un listado de películas que coinciden con el criterio ingresado.

• Caso de Uso 3: Consultar Detalles de Película

Actor: Usuario.

Descripción: Al seleccionar una película específica (ya sea desde el inicio o desde los resultados de búsqueda), el usuario accede a una vista detallada donde puede leer la sinopsis, ver la calificación y otros datos relevantes.

• Caso de Uso 4: Gestión de Favoritos (Guardar/Eliminar)

Actor: Usuario.

Descripción:

▪ Guardar: El usuario marca una película de su interés para añadirla a su lista personal de "Favoritos" y acceder a ella posteriormente.

▪ Eliminar: El usuario accede a su sección de favoritos y desmarca una película para removerla de la lista.

6. DEFINICIÓN DEL STACK TECNOLÓGICO

6.1 Stack Tecnológico

Frontend (Aplicación móvil iOS)
-Lenguaje de programación: Swift
-Entorno de desarrollo: Xcode
-Framework de interfaz: SwiftUI
-Arquitectura interna: Modelo-Vista-Controlador (MVC)
-Consumo de API: HTTP REST
-Formato de datos: JSON

Justificación:
Swift es el lenguaje oficial para el desarrollo nativo en iOS, ofreciendo alto rendimiento y seguridad. SwiftUI permite crear interfaces modernas y mantenibles. Xcode proporciona herramientas integradas como simuladores y depuración avanzada. El desarrollo nativo mejora la optimización y experiencia de usuario en dispositivos Apple.

Backend
-Framework Backend: Flask
-Lenguaje: Python
-Base de Datos: MySQL
-Arquitectura: API RESTful
-Formato de intercambio: JSON

Justificación:
Flask es un framework ligero ideal para construir APIs REST. Python permite rapidez en el desarrollo y mantenimiento del backend. MySQL garantiza integridad referencial y estabilidad. La arquitectura REST desacopla la aplicación móvil del servidor, permitiendo escalabilidad y reutilización del backend.

6.2 Arquitectura Propuesta (MVC)

Modelo (Model)

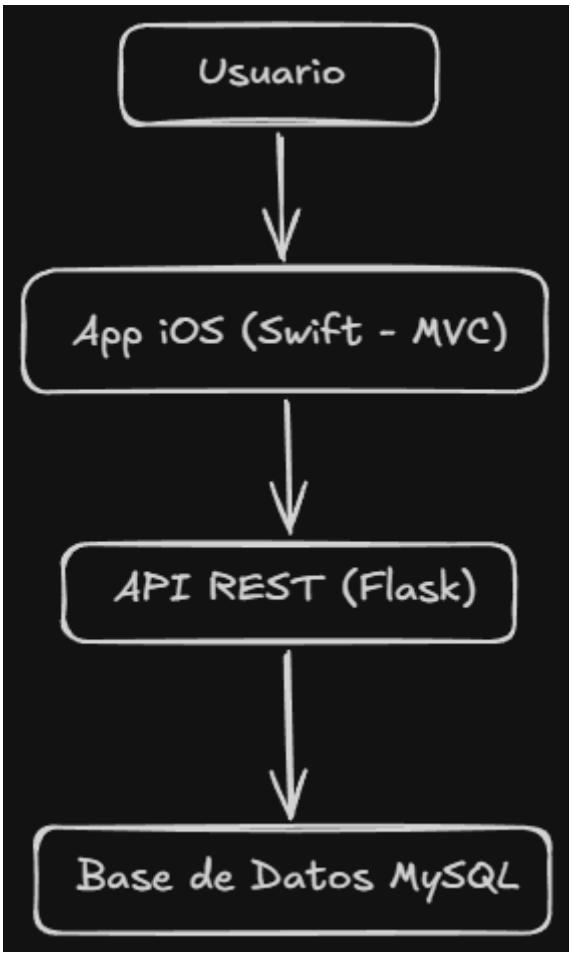
Representa las estructuras de datos como Usuario y demás entidades del sistema. Utiliza Codable para serialización JSON en iOS y modelos ORM en el backend para interactuar con MySQL.

Vista (View)
Interfaces desarrolladas con SwiftUI o Storyboards. Muestra datos y captura la interacción del usuario sin contener lógica de negocio.

Controlador (Controller)
Intermediario entre Vista y Modelo. Gestiona llamadas HTTP al backend, procesa respuestas y actualiza la interfaz controlando estados de carga y error.

6.3 Arquitectura General del Sistema

La arquitectura sigue un modelo Cliente–Servidor:



Ventajas: separación clara de responsabilidades, escalabilidad del servidor, seguridad centralizada en el backend y posibilidad de reutilizar la API para futuras aplicaciones web.

6.4 Persistencia de Datos

Persistencia principal en el backend mediante MySQL, donde se almacenan usuarios, datos de la aplicación, relaciones entre entidades e historial de operaciones.

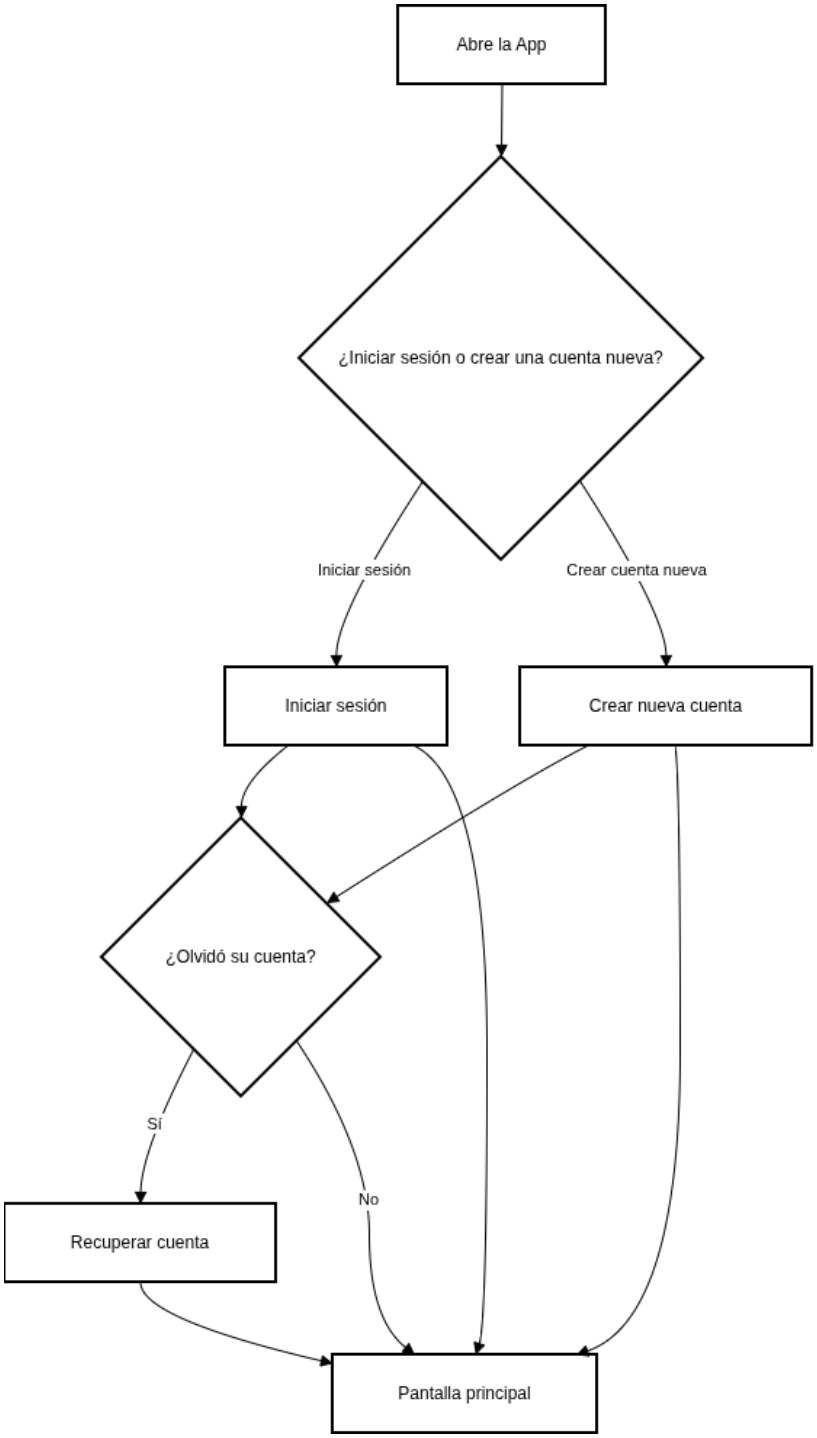
Persistencia local opcional en iOS mediante Core Data o SQLite para modo offline y caché de datos.

7. DISEÑO DE INTERFAZ (UI) Y MAPA DE FUNCIONAMIENTO

DISEÑO DE INTERFAZ (UI)

Inicio de sesión: 	**Pantalla de registro:**
Recuperación de usuario:	**Pantalla principal:**

MAPA DE FUNCIONAMIENTO



BASE DE DATOS

Para guardar la información se utilizará una base de datos relacional. Se eligió este modelo porque permite organizar la información en tablas fijas, lo cual es ideal para asegurar que cada dato esté correctamente enlazado a un usuario específico. Esto garantiza que los datos estén ordenados, no se pierdan o se mezclen entre cuentas.